Containerlab 入門

aoki-taquan

2026 年 03 月 15 日

目次

1 はじめに 5
1.1 Containerlab とは 5
1.2 ユースケース 5
1.3 このガイドについて 5
1.4 対象読者 5
1.5 前提知識 5
2 インストール 7
2.1 動作要件 7
2.2 クイックセットアップ（推奨） 7
2.3 インストールスクリプト 7
2.4 パッケージマネージャ 8
2.4.1 APT（Debian / Ubuntu） 8
2.4.2 YUM / DNF（RHEL / Fedora） 8
2.5 Docker コンテナとして実行 8
2.6 インストールの確認 8
2.7 Docker の確認 8
2.8 アップデート 8
3 基本概念 10
3.1 トポロジファイル 10
3.2 ノード（nodes） 10
3.3 リンク（links） 11
3.4 Kind（ノード種別） 11
3.5 コンテナの命名規則 11
3.6 管理ネットワーク 12
3.7 設定の継承 12
4 クイックスタート 13
4.1 最初のラボを作る 13
4.1.1 トポロジファイルの作成 13
4.1.2 ラボのデプロイ 13
4.1.3 ラボの確認 13
4.2 ノードへの接続 14
4.2.1 SSH でアクセス 14
4.2.2 Docker exec でアクセス 14
4.3 疎通確認 14
4.4 ラボの破棄 14
5 トポロジ定義の詳細 16
5.1 トポロジファイルの全体構造 16
5.2 プレフィックスのカスタマイズ 16
5.3 ノードの詳細設定 16

5.3.1 startup-config ... 16
5.3.2 binds（ボリュームマウント） ... 16
5.3.3 ports（ポートマッピング） ... 17
5.3.4 exec（起動後コマンド） ... 17
5.4 リンクの詳細 ... 17
5.4.1 簡略フォーマット ... 17
5.4.2 拡張フォーマット ... 17
5.4.3 特殊なリンクタイプ ... 18
5.5 インターフェース名のエイリアス ... 18
5.6 マジック変数 ... 18
5.7 環境変数の参照 ... 18
6 CLI リファレンス ... 19
6.1 基本コマンド ... 19
6.1.1 deploy（デプロイ） ... 19
6.1.2 destroy（破棄） ... 19
6.1.3 inspect（検査） ... 19
6.1.4 save（保存） ... 20
6.1.5 graph（グラフ） ... 20
6.1.6 exec（コマンド実行） ... 20
6.1.7 generate（生成） ... 20
6.2 省略形 ... 20
6.3 グローバルフラグ ... 21
7 マルチベンダー環境 ... 22
7.1 複数ベンダーの組み合わせ ... 22
7.1.1 マルチベンダートポロジの例 ... 22
7.2 コンテナベースの NOS ... 22
7.2.1 Nokia SR Linux ... 22
7.2.2 Arista cEOS ... 22
7.2.3 Juniper cRPD ... 23
7.2.4 Cisco XRd ... 23
7.3 Linux コンテナ ... 23
7.4 VM ベースの NOS（vrnetlab） ... 23
7.5 コンテナイメージの取得 ... 24
8 設定管理 ... 25
8.1 startup-config ... 25
8.1.1 ローカルファイルから読み込み ... 25
8.1.2 設定例（SR Linux） ... 25
8.2 設定の保存 ... 26
8.3 ラボディレクトリ ... 26
8.4 Ansible インベントリの自動生成 ... 26
8.5 設定のバージョン管理 ... 27
9 実践的なトポロジ ... 28

9.1 Leaf-Spine トポロジ 28
9.2 ISP ネットワーク構成 28
9.3 テレメトリスタック統合 29
9.4 トラフィックジェネレータの統合 30
9.5 node-filter によるノードの部分デプロイ 30
10 CI/CD との統合 31
10.1 GitHub Actions での利用 31
10.1.1 基本的なワークフロー 31
10.1.2 公式 GitHub Action 31
10.2 GitLab CI での利用 32
10.3 ネットワークテストの自動化 32
10.3.1 Robot Framework との組み合わせ 32
10.3.2 pytest によるテスト 32
10.4 ネットワーク変更の検証パイプライン 33

1 はじめに

1.1 Containerlab とは

Containerlab は、コンテナベースのネットワークラボを構築・管理するためのオープンソース CLI ツールです。YAML ファイルでネットワークトポロジを定義し、コマンド一つでラボ環境を起動できます。 Nokia が開発し、現在はコミュニティで活発にメンテナンスされています。

主な特徴は以下の通りです：

- **Lab as Code**：YAML ファイルでトポロジを宣言的に定義し、Git で管理できる
- **軽量・高速**：コンテナベースのため VM より少ないリソースで大規模トポロジを構築可能
- **マルチベンダー対応**：Nokia SR Linux、Arista cEOS、Cisco XRd、Juniper cRPD など多数の NOS をサポート
- **VM サポート**：vrnetlab 統合により、コンテナ化されていない NOS も VM として利用可能
- **CI/CD 統合**：GitHub Actions や GitLab CI でネットワークテストを自動化できる
- **シンプルな CLI**：`deploy`、`destroy`、`inspect` など直感的なコマンド体系

1.2 ユースケース

Containerlab は以下のような場面で活用されています：

- **ネットワーク設計の検証**：本番投入前にトポロジや設定を安全にテスト
- **ネットワーク自動化の開発**：Ansible、Nornir、Terraform などの自動化ツールの開発・テスト環境
- **テレメトリ検証**：gNMI、Prometheus、Grafana などのモニタリングスタックとの統合テスト
- **教育・トレーニング**：ネットワークエンジニアの学習環境として
- **CI/CD パイプライン**：ネットワーク構成変更の自動テスト
- **相互接続テスト**：マルチベンダー環境での互換性検証

1.3 このガイドについて

本書は Containerlab を初めて使う方を対象に、インストールからトポロジの定義、 ラボの構築・管理、高度な活用法までを解説する入門書です。

1.4 対象読者

- ネットワークエンジニア・ネットワーク自動化エンジニア
- Docker の基本を理解している方
- ネットワークラボ環境を手軽に構築したい方

1.5 前提知識

- 基本的な Linux ターミナル操作（`cd`、`ls` など）

- Docker の基本概念（コンテナ、イメージ）
- ネットワークの基礎知識（IP アドレス、ルーティングの概念）

2 インストール

2.1 動作要件

Containerlab を利用するには以下が必要です：

- **OS**：Linux（Ubuntu 20.04+、Debian 10+、RHEL 8+、Fedora、Rocky Linux）
- **Docker**：Docker CE がインストール済みであること
- **権限**：`sudo` 権限（コンテナネットワーク操作に必要）
- **CPU**：x86_64 または ARM64
- **メモリ**：ラボ規模に依存（最低 4GB 推奨）

プラットフォーム	サポート状況
Linux（ネイティブ）	フルサポート。推奨環境
Windows（WSL2）	WSL2 + Docker で動作可能
macOS	Docker Desktop / Colima で基本機能のみ。vrnetlab 非対応

2.2 クイックセットアップ（推奨）

Docker と Containerlab を一括でインストールするスクリプトが用意されています：

```
curl -sL https://containerlab.dev/setup | sudo -E bash -s "all"
```

このコマンドは以下を実行します：
- Docker CE のインストール（未インストールの場合）
- Containerlab のインストール
- GitHub CLI のインストール

2.3 インストールスクリプト

Docker が既にインストール済みの場合、Containerlab のみをインストールできます：

```
bash -c "$(curl -sL https://get.containerlab.dev)"
```

特定のバージョンを指定する場合：

```
bash -c "$(curl -sL https://get.containerlab.dev)" -- -v 0.60.0
```

2.4 パッケージマネージャ

2.4.1 APT（Debian / Ubuntu）

```
echo "deb [trusted=yes] https://netdevops.fury.site/apt/ /" | \
  sudo tee /etc/apt/sources.list.d/netdevops.list
sudo apt update && sudo apt install containerlab
```

2.4.2 YUM / DNF（RHEL / Fedora）

```
sudo dnf config-manager addrepo \
  --set=baseurl="https://netdevops.fury.site/yum/"
sudo dnf install containerlab
```

2.5 Docker コンテナとして実行

Containerlab 自体をコンテナとして実行することもできます：

```
docker pull ghcr.io/srl-labs/clab
```

2.6 インストールの確認

```
containerlab version
```

バージョン情報が表示されれば正常にインストールされています。

2.7 Docker の確認

Containerlab は Docker を必要とします。Docker が正しく動作しているか確認します：

```
docker info
```

Docker が起動していない場合：

```
sudo systemctl start docker
sudo systemctl enable docker
```

2.8 アップデート

最新版へのアップデートは、インストールスクリプトを再実行するだけです：

```
bash -c "$(curl -sL https://get.containerlab.dev)"
```

パッケージマネージャ経由の場合は通常のアップデートコマンドを使用します：

```
sudo apt update && sudo apt upgrade containerlab  # Debian/Ubuntu
sudo dnf update containerlab                       # RHEL/Fedora
```

3 基本概念

3.1 トポロジファイル

Containerlab の中核となるのがトポロジ定義ファイル（`.clab.yml`）です。 YAML 形式でネットワークラボの構成を宣言的に記述します。

基本的な構造は以下の通りです：

```yaml
name: my-lab

topology:
  nodes:
    router1:
      kind: nokia_srlinux
      image: ghcr.io/nokia/srlinux:24.10
    router2:
      kind: nokia_srlinux
      image: ghcr.io/nokia/srlinux:24.10

  links:
    - endpoints: ["router1:e1-1", "router2:e1-1"]
```

3.2 ノード（nodes）

ノードはラボ内の各ネットワーク機器を表します。ノードには以下のプロパティがあります：

- `kind`：ノードの種類（NOS タイプ）
- `image`：使用するコンテナイメージ
- `type`：ハードウェアモデル（kind に応じて）
- `startup-config`：起動時に適用する設定ファイル
- `binds`：ホストとコンテナ間のボリュームマウント
- `ports`：ポートマッピング
- `env`：環境変数
- `labels`：メタデータラベル
- `exec`：起動後に実行するコマンド

```yaml
nodes:
  srl:
    kind: nokia_srlinux
    image: ghcr.io/nokia/srlinux:24.10
    startup-config: configs/srl.cfg
    ports:
      - "8080:80"
```

```
    env:
      MY_VAR: "value"
```

3.3 リンク（links）

リンクはノード間の接続を定義します。基本フォーマットはエンドポイントのペアです：

```
links:
  - endpoints: ["node1:e1-1", "node2:e1-1"]
```

IP アドレスを直接割り当てることもできます：

```
links:
  - endpoints: ["node1:e1-1", "node2:e1-1"]
    ipv4: ["192.168.1.1/24", "192.168.1.2/24"]
```

3.4 Kind（ノード種別）

`kind` はノードの NOS タイプを指定します。主要な kind は以下の通りです：

ベンダー	kind	説明
Nokia	`nokia_srlinux`	SR Linux コンテナ
Arista	`arista_ceos`	cEOS コンテナ
Cisco	`cisco_xrd`	XRd コンテナ
Cisco	`cisco_iol`	IOL コンテナ
Juniper	`juniper_crpd`	cRPD コンテナ
Cumulus	`cumulus_cvx`	Cumulus VX
SONiC	`sonic-vs`	SONiC 仮想スイッチ
Linux	`linux`	汎用 Linux コンテナ

3.5 コンテナの命名規則

Containerlab はコンテナに以下の命名規則を適用します：

```
clab-<lab-name>-<node-name>
```

例えば、ラボ名が `my-lab`、ノード名が `router1` の場合：

```
clab-my-lab-router1
```

この名前で `docker exec` や SSH アクセスが可能です。

3.6 管理ネットワーク

Containerlab はデフォルトで管理ネットワーク（`clab` ブリッジ）を作成し、すべてのノードに管理インターフェースを接続します。 これにより、ホストから各ノードへ SSH でアクセスできます。

3.7 設定の継承

トポロジファイルでは設定の継承がサポートされています。優先順位は以下の通りです（高い順）：

1. ノード固有の設定
2. グループの設定
3. Kind の設定
4. デフォルトの設定

```yaml
topology:
  defaults:
    env:
      COMMON_VAR: "shared"

  kinds:
    nokia_srlinux:
      image: ghcr.io/nokia/srlinux:24.10

  nodes:
    srl1:
      kind: nokia_srlinux
    srl2:
      kind: nokia_srlinux
```

この例では `srl1` と `srl2` の両方が同じイメージを使用します。

4 クイックスタート

4.1 最初のラボを作る

実際にラボを構築してみましょう。Nokia SR Linux 2 台を接続するシンプルなトポロジを作成します。

4.1.1 トポロジファイルの作成

作業ディレクトリを作成し、トポロジファイルを記述します：

```
mkdir ~/first-lab && cd ~/first-lab
```

以下の内容で `first-lab.clab.yml` を作成します：

```
name: first-lab

topology:
  nodes:
    srl1:
      kind: nokia_srlinux
      image: ghcr.io/nokia/srlinux:24.10
    srl2:
      kind: nokia_srlinux
      image: ghcr.io/nokia/srlinux:24.10

  links:
    - endpoints: ["srl1:e1-1", "srl2:e1-1"]
```

4.1.2 ラボのデプロイ

```
sudo containerlab deploy
```

Containerlab はカレントディレクトリ内の `*.clab.yml` ファイルを自動検出します。 ファイルを明示的に指定する場合は `--topo` フラグを使用します：

```
sudo containerlab deploy --topo first-lab.clab.yml
```

デプロイが完了すると、ノード情報の一覧が表示されます。

4.1.3 ラボの確認

デプロイされたラボの状態を確認します：

```
sudo containerlab inspect
```

すべてのラボを一覧表示する場合：

```
sudo containerlab inspect --all
```

4.2 ノードへの接続

4.2.1 SSH でアクセス

SR Linux のデフォルト認証情報は `admin` / `NokiaSrl1!` です：

```
ssh admin@clab-first-lab-srl1
```

4.2.2 Docker exec でアクセス

```
docker exec -it clab-first-lab-srl1 sr_cli
```

4.3 疎通確認

srl1 から srl2 へインターフェースを設定し、ping で疎通を確認してみましょう。

srl1 の CLI で：

```
set / interface ethernet-1/1 admin-state enable
set / interface ethernet-1/1 subinterface 0 \
    ipv4 admin-state enable \
    address 192.168.1.1/24
commit now
```

srl2 の CLI で：

```
set / interface ethernet-1/1 admin-state enable
set / interface ethernet-1/1 subinterface 0 \
    ipv4 admin-state enable \
    address 192.168.1.2/24
commit now
```

srl1 から ping：

```
ping 192.168.1.2 network-instance default
```

4.4 ラボの破棄

ラボが不要になったら破棄します：

```
sudo containerlab destroy
```

特定のトポロジファイルを指定する場合：

```
sudo containerlab destroy --topo first-lab.clab.yml
```

`--cleanup` フラグでラボディレクトリも削除できます：

```
sudo containerlab destroy --cleanup
```

5 トポロジ定義の詳細

5.1 トポロジファイルの全体構造

```
name: <ラボ名>
prefix: <プレフィックス>
topology:
  defaults: {}
  kinds: {}
  nodes: {}
  links: []
```

5.2 プレフィックスのカスタマイズ

コンテナ名のプレフィックスを変更できます：

設定	例（ラボ名: lab, ノード: r1）	説明
未指定	`clab-lab-r1`	デフォルト
`prefix: mylab`	`mylab-lab-r1`	カスタムプレフィックス
`prefix: __lab-name`	`lab-r1`	プレフィックスなし
`prefix: ""`	`r1`	ラボ名もプレフィックスも省略

5.3 ノードの詳細設定

5.3.1 startup-config

ノードの起動設定を外部ファイルから読み込めます：

```
nodes:
  srl:
    kind: nokia_srlinux
    image: ghcr.io/nokia/srlinux:24.10
    startup-config: configs/srl-config.json
```

5.3.2 binds（ボリュームマウント）

ホストのファイルやディレクトリをコンテナにマウントします：

```
nodes:
  linux1:
    kind: linux
    image: alpine:latest
```

```
    binds:
      - /host/path:/container/path:ro
      - configs/test.sh:/test.sh
```

5.3.3 ports（ポートマッピング）

ホストからコンテナのポートにアクセスするための設定です：

```
nodes:
  srl:
    kind: nokia_srlinux
    image: ghcr.io/nokia/srlinux:24.10
    ports:
      - "8080:80"
      - "9443:443"
```

5.3.4 exec（起動後コマンド）

ノード起動後に自動実行するコマンドを指定します：

```
nodes:
  linux1:
    kind: linux
    image: alpine:latest
    exec:
      - ip addr add 10.0.0.1/24 dev eth1
      - ip route add 10.0.1.0/24 via 10.0.0.254
```

5.4 リンクの詳細

5.4.1 簡略フォーマット

最も一般的な形式です：

```
links:
  - endpoints: ["srl1:e1-1", "srl2:e1-1"]
```

5.4.2 拡張フォーマット

MAC アドレスや MTU を指定する場合に使用します：

```
links:
  - type: veth
    endpoints:
      - node: srl1
```

```
        interface: e1-1
        mac: "aa:bb:cc:dd:ee:01"
      - node: srl2
        interface: e1-1
        mac: "aa:bb:cc:dd:ee:02"
    mtu: 9000
```

5.4.3 特殊なリンクタイプ

タイプ	説明
`veth`	デフォルト。仮想イーサネットペア
`mgmt-net`	管理ネットワークブリッジへの接続
`macvlan`	ホストインターフェース上の MACVlan
`host`	コンテナとホスト間の veth ペア
`vxlan`	VXLAN トンネルインターフェース

5.5 インターフェース名のエイリアス

各 NOS のインターフェース命名規則に対応したエイリアスが使えます：

NOS	エイリアス例
SR Linux	`ethernet-1/1` → `e1-1`
Arista cEOS	`Ethernet1/1` → `eth1`
Linux	`eth1`（そのまま）

5.6 マジック変数

トポロジファイル内で使用できる特殊変数です：

- `__clabLabName__`：ラボ名
- `__clabNodeName__`：ノード名
- `__clabNodeDir__`：ノードディレクトリのパス
- `__clabDir__`：ラボディレクトリのパス

5.7 環境変数の参照

トポロジファイル内で環境変数を参照できます：

```
nodes:
  srl:
    kind: nokia_srlinux
    image: ${SRL_IMAGE:-ghcr.io/nokia/srlinux:24.10}
```

`${VAR:-default}` 構文でデフォルト値を指定できます。

6 CLI リファレンス

6.1 基本コマンド

Containerlab のコマンドは直感的で、ラボのライフサイクル管理に必要な操作を網羅しています。

6.1.1 deploy（デプロイ）

ラボを起動します：

```
sudo containerlab deploy                          # 自動検出
sudo containerlab deploy --topo lab.clab.yml      # ファイル指定
sudo containerlab deploy --reconfigure            # 既存ラボを再構成
```

主なフラグ：

フラグ	説明
`--topo`, `-t`	トポロジファイルのパス
`--reconfigure`	既存のラボを再構成
`--max-workers`	並列起動するノード数
`--export-template`	トポロジデータのエクスポートテンプレート
`--node-filter`	特定のノードのみデプロイ

6.1.2 destroy（破棄）

ラボを停止し削除します：

```
sudo containerlab destroy                         # 自動検出
sudo containerlab destroy --topo lab.clab.yml     # ファイル指定
sudo containerlab destroy --cleanup               # ラボディレクトリも削除
sudo containerlab destroy --all                   # すべてのラボを破棄
```

6.1.3 inspect（検査）

ラボの状態を確認します：

```
sudo containerlab inspect                         # 現在のラボ
sudo containerlab inspect --all                   # すべてのラボ
sudo containerlab inspect --format json           # JSON 形式で出力
```

出力には以下の情報が含まれます：

- ノード名
- コンテナ ID
- イメージ
- Kind

- 状態（running / stopped）
- IPv4 / IPv6 アドレス

6.1.4 save（保存）

実行中のノードの設定を保存します：

```
sudo containerlab save
sudo containerlab save --topo lab.clab.yml
```

NOS がサポートしている場合、ノードの現在のコンフィグをファイルに保存します。

6.1.5 graph（グラフ）

トポロジの可視化を行います：

```
sudo containerlab graph
sudo containerlab graph --topo lab.clab.yml
```

デフォルトで Web ブラウザでインタラクティブなトポロジ図を表示します。

6.1.6 exec（コマンド実行）

すべてのノードまたは特定のノードでコマンドを実行します：

```
sudo containerlab exec --topo lab.clab.yml \
  --label clab-node-name=srl1 \
  --cmd "ip addr show"
```

6.1.7 generate（生成）

設定ファイルの自動生成を行います：

```
sudo containerlab generate --name my-lab \
  --kind nokia_srlinux \
  --nodes 4 \
  --deploy
```

6.2 省略形

Containerlab は短縮コマンドにも対応しています：

```
clab deploy    # containerlab deploy の省略形
clab destroy   # containerlab destroy の省略形
clab inspect   # containerlab inspect の省略形
```

6.3 グローバルフラグ

すべてのコマンドで使用可能なフラグです：

フラグ	説明
`--runtime`	コンテナランタイムの指定（docker / podman）
`--debug`	デバッグログを出力
`--log-level`	ログレベルの指定
`--timeout`	操作のタイムアウト時間

7 マルチベンダー環境

7.1 複数ベンダーの組み合わせ

Containerlab の大きな強みは、異なるベンダーの NOS を一つのラボ内で組み合わせられることです。

7.1.1 マルチベンダートポロジの例

Nokia SR Linux と Arista cEOS を接続する例：

```
name: multivendor-lab

topology:
  nodes:
    srl:
      kind: nokia_srlinux
      image: ghcr.io/nokia/srlinux:24.10
    ceos:
      kind: arista_ceos
      image: ceos:4.32.0F

  links:
    - endpoints: ["srl:e1-1", "ceos:eth1"]
```

7.2 コンテナベースの NOS

7.2.1 Nokia SR Linux

オープンで無料のコンテナ NOS です。Containerlab で最もよく使われます：

```
nodes:
  srl:
    kind: nokia_srlinux
    image: ghcr.io/nokia/srlinux:24.10
    type: ixrd3l  # ハードウェアタイプ
```

デフォルト認証情報：`admin` / `NokiaSrl1!`

7.2.2 Arista cEOS

Arista の EOS をコンテナ化したものです。arista.com からイメージを取得する必要があります：

```
nodes:
  ceos:
```

```
    kind: arista_ceos
    image: ceos:4.32.0F
```

デフォルト認証情報：`admin`（パスワードなし）

7.2.3 Juniper cRPD

Juniper のルーティングプロトコルデーモンをコンテナ化したものです：

```
nodes:
  crpd:
    kind: juniper_crpd
    image: crpd:23.4R1.10
```

7.2.4 Cisco XRd

Cisco IOS XR のコンテナ版です：

```
nodes:
  xrd:
    kind: cisco_xrd
    image: ios-xr/xrd-control-plane:24.1.1
```

7.3 Linux コンテナ

汎用の Linux コンテナをホストやクライアントとして利用できます：

```
nodes:
  client1:
    kind: linux
    image: alpine:latest
    exec:
      - ip addr add 10.0.0.10/24 dev eth1
      - ip route add default via 10.0.0.1

  server1:
    kind: linux
    image: nginx:latest
    ports:
      - "8080:80"
```

7.4 VM ベースの NOS（vrnetlab）

コンテナ化されていない NOS は vrnetlab 統合により VM として実行できます。主な対応 NOS は以下の通りです：

NOS	kind
Juniper vMX	`juniper_vmx`
Juniper vSRX	`juniper_vsrx`
Cisco CSR1000v	`cisco_csr1000v`
Cisco Nexus 9000v	`cisco_n9kv`
Aruba AOS-CX	`aruba_aoscx`
Palo Alto PAN-OS	`paloalto_panos`

vrnetlab を使用するには、ベンダーから NOS イメージを取得し、 vrnetlab のビルドスクリプトでコンテナイメージを作成する必要があります。

7.5 コンテナイメージの取得

各ベンダーのイメージ取得方法は異なります：

- **Nokia SR Linux**：GitHub Container Registry から無料で取得可能（`ghcr.io/nokia/srlinux`）
- **Arista cEOS**：arista.com でアカウント登録後にダウンロード
- **Juniper cRPD**：Juniper のサイトから取得（評価版あり）
- **Cisco XRd**：Cisco のソフトウェアダウンロードから取得

8 設定管理

8.1 startup-config

ノードにあらかじめ設定を投入した状態でラボを起動できます。

8.1.1 ローカルファイルから読み込み

```yaml
name: config-lab

topology:
  nodes:
    srl:
      kind: nokia_srlinux
      image: ghcr.io/nokia/srlinux:24.10
      startup-config: configs/srl-config.json
    ceos:
      kind: arista_ceos
      image: ceos:4.32.0F
      startup-config: configs/ceos-config.cfg
```

設定ファイルの形式は NOS ごとに異なります：

- **SR Linux**：JSON 形式
- **Arista cEOS**：EOS CLI 形式
- **Juniper cRPD**：Junos CLI 形式

8.1.2 設定例（SR Linux）

```json
{
  "interface": [
    {
      "name": "ethernet-1/1",
      "admin-state": "enable",
      "subinterface": [
        {
          "index": 0,
          "ipv4": {
            "admin-state": "enable",
            "address": [
              {
                "ip-prefix": "192.168.1.1/24"
              }
            ]
          }
        }
      ]
```

```
    }
  ]
}
```

8.2 設定の保存

実行中のラボの設定を保存できます：

```
sudo containerlab save
```

保存された設定はラボディレクトリ内の各ノードディレクトリに格納されます：

```
clab-<lab-name>/
  <node-name>/
    config/
      ...  # NOS 固有の設定ファイル
```

8.3 ラボディレクトリ

ラボをデプロイすると、トポロジファイルと同じ場所にラボディレクトリが作成されます：

```
clab-<lab-name>/
  topology-data.json    # トポロジメタデータ
  ansible-inventory.yml # Ansible インベントリ
  <node-name>/          # 各ノードのディレクトリ
    config/             # 設定ファイル
    tls/                # TLS 証明書（対応ノードのみ）
```

8.4 Ansible インベントリの自動生成

Containerlab はデプロイ時に Ansible インベントリファイルを自動生成します：

```
ansible-playbook -i clab-my-lab/ansible-inventory.yml playbook.yml
```

生成されるインベントリの構造：

```
all:
  children:
    nokia_srlinux:
      hosts:
        clab-my-lab-srl1:
          ansible_host: <management-ip>
    arista_ceos:
      hosts:
```

```
        clab-my-lab-ceos1:
          ansible_host: <management-ip>
```

8.5 設定のバージョン管理

startup-config ファイルを Git で管理することで、ラボの設定変更を追跡できます：

```
my-lab/
  lab.clab.yml
  configs/
    srl1.json
    srl2.json
    ceos1.cfg
  README.md
```

この構成により、トポロジと設定を一元的にバージョン管理できます。

9 実践的なトポロジ

9.1 Leaf-Spine トポロジ

データセンターで一般的な Leaf-Spine 構成の例です：

```yaml
name: leaf-spine

topology:
  kinds:
    nokia_srlinux:
      image: ghcr.io/nokia/srlinux:24.10

  nodes:
    spine1:
      kind: nokia_srlinux
    spine2:
      kind: nokia_srlinux
    leaf1:
      kind: nokia_srlinux
    leaf2:
      kind: nokia_srlinux
    leaf3:
      kind: nokia_srlinux

  links:
    # Spine1 - Leaf 接続
    - endpoints: ["spine1:e1-1", "leaf1:e1-49"]
    - endpoints: ["spine1:e1-2", "leaf2:e1-49"]
    - endpoints: ["spine1:e1-3", "leaf3:e1-49"]
    # Spine2 - Leaf 接続
    - endpoints: ["spine2:e1-1", "leaf1:e1-50"]
    - endpoints: ["spine2:e1-2", "leaf2:e1-50"]
    - endpoints: ["spine2:e1-3", "leaf3:e1-50"]
```

9.2 ISP ネットワーク構成

BGP ピアリングを含む ISP 風のトポロジ例：

```yaml
name: isp-network

topology:
  kinds:
    nokia_srlinux:
      image: ghcr.io/nokia/srlinux:24.10
```

```yaml
  nodes:
    core1:
      kind: nokia_srlinux
      startup-config: configs/core1.json
    core2:
      kind: nokia_srlinux
      startup-config: configs/core2.json
    edge1:
      kind: nokia_srlinux
      startup-config: configs/edge1.json
    edge2:
      kind: nokia_srlinux
      startup-config: configs/edge2.json
    client:
      kind: linux
      image: alpine:latest

  links:
    - endpoints: ["core1:e1-1", "core2:e1-1"]
    - endpoints: ["core1:e1-2", "edge1:e1-1"]
    - endpoints: ["core2:e1-2", "edge2:e1-1"]
    - endpoints: ["edge1:e1-2", "client:eth1"]
```

9.3 テレメトリスタック統合

gNMI テレメトリと Grafana を組み合わせた監視環境の例：

```yaml
name: telemetry-lab

topology:
  nodes:
    srl1:
      kind: nokia_srlinux
      image: ghcr.io/nokia/srlinux:24.10
    srl2:
      kind: nokia_srlinux
      image: ghcr.io/nokia/srlinux:24.10

    gnmic:
      kind: linux
      image: ghcr.io/openconfig/gnmic:latest
      binds:
        - gnmic-config.yml:/app/gnmic.yml:ro
      env:
        GNMIC_API: ":7890"

    prometheus:
      kind: linux
```

```
      image: prom/prometheus:latest
      binds:
        - prometheus.yml:/etc/prometheus/prometheus.yml:ro
      ports:
        - "9090:9090"

    grafana:
      kind: linux
      image: grafana/grafana:latest
      ports:
        - "3000:3000"

  links:
    - endpoints: ["srl1:e1-1", "srl2:e1-1"]
```

9.4 トラフィックジェネレータの統合

ixia-c（Keysight の OTG 準拠トラフィックジェネレータ）との統合例：

```
name: traffic-test

topology:
  nodes:
    srl:
      kind: nokia_srlinux
      image: ghcr.io/nokia/srlinux:24.10
    ixia:
      kind: keysight_ixia-c-one
      image: ghcr.io/open-traffic-generator/ixia-c-one:latest

  links:
    - endpoints: ["ixia:eth1", "srl:e1-1"]
    - endpoints: ["ixia:eth2", "srl:e1-2"]
```

9.5 node-filter によるノードの部分デプロイ

大規模トポロジの一部だけをデプロイしたい場合に便利です：

```
sudo containerlab deploy --node-filter "spine1,leaf1,leaf2"
```

テスト対象のノードだけを起動することで、リソースを節約できます。

10 CI/CD との統合

10.1 GitHub Actions での利用

Containerlab はシングルバイナリで動作するため、CI/CD パイプラインとの統合が容易です。

10.1.1 基本的なワークフロー

```yaml
name: Network Test

on:
  push:
    branches: [main]
  pull_request:
    branches: [main]

jobs:
  network-test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - name: Install containerlab
        run: |
          bash -c "$(curl -sL https://get.containerlab.dev)"

      - name: Deploy lab
        run: |
          sudo containerlab deploy --topo lab.clab.yml

      - name: Run tests
        run: |
          # ノードへの疎通確認
          docker exec clab-lab-srl1 \
            ip netns exec srbase ping -c 3 192.168.1.2
          # 自動化テストの実行
          python -m pytest tests/

      - name: Destroy lab
        if: always()
        run: |
          sudo containerlab destroy --topo lab.clab.yml
```

10.1.2 公式 GitHub Action

Containerlab は公式の GitHub Action も提供しています：

```yaml
- name: Deploy containerlab
  uses: srl-labs/containerlab-action@v1
  with:
    topo: lab.clab.yml
```

10.2 GitLab CI での利用

```yaml
network-test:
  image: docker:latest
  services:
    - docker:dind
  before_script:
    - bash -c "$(curl -sL https://get.containerlab.dev)"
  script:
    - containerlab deploy --topo lab.clab.yml
    - python -m pytest tests/
  after_script:
    - containerlab destroy --topo lab.clab.yml
```

10.3 ネットワークテストの自動化

10.3.1 Robot Framework との組み合わせ

```bash
# ラボデプロイ後にテストを実行
sudo containerlab deploy --topo lab.clab.yml
robot --outputdir results tests/
sudo containerlab destroy --topo lab.clab.yml
```

10.3.2 pytest によるテスト

```python
import subprocess

def test_bgp_neighbor():
    """BGP ネイバーが確立されていることを確認"""
    result = subprocess.run(
        ["docker", "exec", "clab-lab-srl1",
         "sr_cli", "show", "network-instance",
         "default", "protocols", "bgp", "neighbor"],
        capture_output=True, text=True
    )
    assert "established" in result.stdout.lower()

def test_reachability():
    """エンドツーエンドの疎通を確認"""
```

```
    result = subprocess.run(
        ["docker", "exec", "clab-lab-client",
         "ping", "-c", "3", "10.0.0.1"],
        capture_output=True, text=True
    )
    assert result.returncode == 0
```

10.4 ネットワーク変更の検証パイプライン

ネットワーク設定の変更を PR ベースで管理し、自動テストで検証するフローです：

1. エンジニアが設定ファイルを変更し PR を作成
2. CI がラボをデプロイし、変更後の設定を適用
3. 自動テスト（疎通確認、BGP 状態、ルーティングテーブル検証）を実行
4. テスト合格後にマージ、本番環境に適用

このアプローチにより、ネットワーク変更の品質を自動的に担保できます。